

BAHASA PEMROGRAMAN INDONESIA

Dasar & Fondasi Menguasai KARSA

Modul pertama dari seri pembelajaran KARSA v0.3.1. Pelajari filosofi desain bahasa pemrograman web berbahasa Indonesia, arsitektur 5 tahap pipeline, deklarasi variabel reaktif, alur kontrol, fungsi, penanganan error, hingga perakitan aplikasi kalkulator web interaktif pertama Anda.

● Level Pemula

● 4–6 Jam Belajar

● 10 Bab + Mini-Proyek

Daftar Isi

PENGANTAR

Pengantar Modul & Peta Belajar	3
--------------------------------	---

BAB UTAMA

Bab 1 — Mengenal KARSA & Filosofi Desain	4
Bab 2 — Instalasi & Setup Lingkungan Belajar	6
Bab 3 — Variabel & Tipe Data Dasar	8
Bab 4 — Operator & Struktur Percabangan	11
Bab 5 — Struktur Perulangan & Kontrol Loop	14
Bab 6 — Menulis Fungsi & Interoperabilitas JavaScript	17
Bab 7 — Output DOM & Penciptaan Elemen dengan buat	19
Bab 8 — Interaktivitas Event Handling dengan ketika	22
Bab 9 — Komentar, Dokumentasi & Membedah Pesan Error	24
Bab 10 — Mini-Proyek: Kalkulator Interaktif Lengkap	28

LATIHAN & REFERENSI

Latihan Soal Uji Pemahaman	33
Solusi Latihan & Pembahasan Lengkap	36
Lampiran — Referensi Padanan Sintaks KARSA vs Vanilla JS	39
Checklist Pembelajaran Modul 1	40

PENGANTAR MODUL

Pengantar & Peta Belajar Modul 1

Selamat datang di **Modul 1: Dasar & Fondasi KARSA**. Dokumen ini menggabungkan seluruh materi fundamental pembelajaran KARSA v0.3.1 yang dirancang khusus untuk membawa Anda dari nol hingga berhasil menciptakan aplikasi web interaktif pertama Anda yang berjalan langsung di dalam browser.

KARSA adalah bahasa pemrograman web berbahasa Indonesia yang mengubah kode `.ks` (KARSA Script) menjadi JavaScript DOM API murni — tanpa Virtual DOM, tanpa framework berat, dan tanpa ketergantungan eksternal.

Tujuan Pembelajaran Modul 1

Setelah menyelesaikan modul ini, Anda akan mampu:

1. **Memahami filosofi dan arsitektur KARSA** serta cara kerja 5 tahap pipeline kompilasinya.
2. **Melakukan setup lingkungan pengembangan** yang zero-config di browser (metode inline maupun file `.ks`).
3. **Mendeklarasikan variabel secara tepat** (data, tetap, ubah) dan menguasai tipe data dasar.
4. **Menggunakan operator** aritmatika, perbandingan, dan logika untuk manipulasi data.
5. **Menerapkan struktur alur kontrol** percabangan (jika, kalau, jika tidak) dan perulangan (ulangi, selama).
6. **Membuat dan memanggil fungsi** dengan parameter serta interoperabilitas JavaScript murni.
7. **Menulis komentar** dan dokumentasi kode terstruktur (DocString Premium).
8. **Membuat elemen DOM** menggunakan perintah buat dan menangani event dengan ketika.
9. **Memahami dan mendebug error** (E1xxx hingga E6xxx) secara efisien.
10. **Membangun aplikasi web interaktif** pertama Anda yaitu Kalkulator Interaktif.

◆ Peta Konsep Modul 1 (Mind Map)

1. Filosofi & Pipeline

Lexer, Parser, Compiler

2. Setup & Instalasi

Inline vs File `.ks`

3. Variabel & Data

data, tetap, ubah

4. Operator & Alur

Aritmatika, Percabangan

5. Perulangan

ulangi, selama

6. Fungsi & Interop

Kembalikan, Console.log

7. Output & DOM

tampilkan, buat, 20+ tag

8. Event Handling

ketika, diklik, diketik

9. Error & Doc

E1xxx-E5xxx, DocString

10. Mini-Proyek Lengkap: Kalkulator Interaktif

BAB 1

Mengenal KARSA & Filosofi Desain

1.1 Apa Itu KARSA?

KARSA adalah bahasa pemrograman web yang dirancang khusus untuk para pengembang web di Indonesia menggunakan kata kunci (keywords) berbahasa Indonesia. Dibangun di atas ekosistem JavaScript modern, KARSA memungkinkan pemrogram menulis kode dengan logika yang sangat alami dalam bahasa ibu mereka sendiri, namun menghasilkan output JavaScript vanilla yang 100% kompatibel dengan semua browser modern.

Nama “**KARSA**” diambil dari bahasa Jawa kuno yang berarti “**niat**” atau “**kemauan keras**”. Nama ini mencerminkan semangat juang untuk menciptakan solusi teknologi yang aksesibel, inklusif, dan mudah dipahami oleh seluruh lapisan masyarakat Indonesia, dari pelajar hingga profesional.

1.2 Filosofi Desain KARSA

KARSA dibangun dengan memegang teguh empat pilar filosofi utama:

1. **Zero-Configuration**: Belajar pemrograman web tidak boleh terhambat oleh proses setup yang rumit seperti instalasi Node.js, Webpack, atau konfigurasi compiler. Dalam KARSA, cukup sertakan satu file `karsa.js` di dalam HTML, dan Anda sudah bisa mulai coding.
2. **Sintaksis Alami Berbahasa Indonesia**: Setiap perintah dalam KARSA dirancang menyerupai struktur kalimat bahasa Indonesia baku sehari-hari. Logika pemecahan masalah dapat langsung diekspresikan tanpa hambatan translasi bahasa asing.
3. **Murni Menerjemahkan, Bukan Menjahaj (No Virtual DOM)**: Berbeda dengan framework berat seperti React atau Vue yang menggunakan Virtual DOM, KARSA mengompilasi kode menjadi manipulasi DOM murni (Vanilla JS DOM API). Hal ini menghasilkan performa aplikasi yang sangat cepat, ringan, dan hemat memori.
4. **Ekosistem JavaScript Terbuka**: Anda dapat dengan mudah menggunakan pustaka JavaScript pihak ketiga (third-party JS libraries) di dalam kode KARSA secara mulus tanpa memerlukan sistem interop yang rumit.

1.3 Arsitektur 5 Tahap Pipeline Internal

Ketika Anda menulis kode KARSA dan menjalankannya di dalam browser, kode sumber Anda akan diproses melalui lima tahapan terstruktur yang disebut **“Pipeline KARSA”**.

Setiap tahap memiliki tanggung jawab spesifik dan sistem kode error-nya sendiri. Memahami arsitektur pipeline ini sangat penting agar Anda dapat mengetahui dari mana suatu pesan error berasal dan bagaimana memperbaikinya.

Tahap	Proses Internal	Hasil Output	Deskripsi Tanggung Jawab & Rentang Error
1. Lexer	Tokenization	Deretan Tokens	Memecah teks kode sumber menjadi potongan bermakna (kata kunci, identifier, operator, string). (Error E1xxx)
2. Parser	AST Generation	Abstract Syntax Tree	Menyusun deretan token menjadi pohon hierarki sintaksis sesuai kaidah tata bahasa KARSA. (Error E2xxx)
3. Resolver	Scope Reference	Resolved AST	Memetakan deklarasi variabel, konstanta, dan fungsi serta memeriksa konsistensi cakupan (scope). (Error E3xxx)
4. Analyzer	Type & Lifecycle	Validated AST	Melakukan validasi kesesuaian tipe data (type checking), aturan reaktivitas, dan lifecycle hooks . (Error E4xxx)
5. Compiler	Code Generation	Vanilla JavaScript	Mengubah AST yang telah tervalidasi menjadi kode JavaScript DOM API murni yang siap dieksekusi. (Error E5xxx)

Ilustrasi Alur Kerja Pipeline

Sebagai contoh, jika Anda menulis kode sederhana: `data angka = 10`

- **Lexer** mengubahnya menjadi token: `[KW_DATA, IDENT("angka"), OP_ASSIGN, NUMBER(10)]`.
- **Parser** menyusunnya menjadi struktur pohon penugasan reaktif.
- **Resolver** memverifikasi bahwa nama `angka` belum dipakai di cakupan saat ini.
- **Analyzer** memeriksa bahwa `angka` diinisialisasi dengan nilai bertipe angka.
- **Compiler** mengubahnya menjadi JavaScript murni: `const angka = __createReactive(10);`.

BAB 2

Instalasi & Setup Lingkungan Belajar

2.1 Persiapan Zero-Configuration

Seperti yang dijelaskan pada pilar filosofi desain, KARSA dirancang untuk zero-config. Anda tidak memerlukan instalasi aplikasi khusus di komputer. Anda hanya butuh dua hal yang sudah pasti ada di laptop Anda:

1. **Aplikasi Teks Editor:** Seperti Visual Studio Code, Sublime Text, Notepad++, atau bahkan Notepad biasa.
2. **Web Browser Modern:** Seperti Google Chrome, Mozilla Firefox, Microsoft Edge, atau Safari.

Untuk mengaktifkan engine KARSA, Anda hanya perlu menyisipkan satu baris tag `<script src="engine/karsa.js"></script>` di dalam HTML Anda. Terdapat dua metode penulisan kode yang dapat Anda pilih.

2.2 Metode 1: Penulisan Inline di HTML (Cara Cepat)

Metode ini sangat cocok untuk tahap latihan singkat atau eksperimen cepat. Anda menulis kode KARSA langsung di dalam tag `<script type="text/karsa">` pada HTML Anda.

index.html (Inline Mode)

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aplikasi KARSA Inline</title>
</head>
<body>
  <!-- 1. Sertakan Engine KARSA -->
  <script src="engine/karsa.js"></script>

  <!-- 2. Tulis Kode KARSA Langsung (Inline) -->
  <script type="text/karsa">
    buat h1 -> teks: "Halo, KARSA!"
    buat p -> teks: "Aplikasi pertama berbasis penulisan inline berhasil berjalan."
  </script>
</body>
</html>
```

2.3 Metode 2: Menggunakan File Eksternal .ks (Rekomendasi)

Untuk membangun aplikasi yang rapi, terstruktur, dan berskala besar, sangat disarankan untuk memisahkan kode HTML dengan kode KARSA Anda ke dalam file berekstensi .ks (KARSA Script).

index.html (External Mode)

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Aplikasi KARSA Eksternal</title>
</head>
<body>
  <!-- 1. Sertakan Engine KARSA -->
  <script src="engine/karsa.js"></script>

  <!-- 2. Panggil File Eksternal berekstensi .ks -->
  <script type="text/karsa" src="app.ks"></script>
</body>
</html>
```

app.ks

```
--! Ini adalah isi dari file app.ks
buat h1#judulUtama -> teks: "Selamat Datang di Dunia KARSA!"
buat p.deskripsi -> teks: "Kode ini dipanggil secara bersih dari file eksternal .ks"
buat tombol#tombolSapa -> teks: "Klik Sapaan"

ketika #tombolSapa diklik:
  perbarui #judulUtama -> teks: "Halo dari File Eksternal!"
```

2.4 Struktur Proyek Rekomendasi

Agar proyek web Anda terorganisasi dengan sangat profesional, gunakan susunan direktori berikut:

Rekomendasi Struktur Folder

```
proyek-karsa-anda/
├── index.html      # Halaman utama (Entry point)
├── app.ks          # File logika KARSA utama
└──
```



```
| engine/
|   └─ karsa.js      # Pustaka Engine Kompilator KARSA (v0.3.1)
|
|   └─ gaya/
|       └─ style.css  # File CSS (Opsional) untuk styling manual
```

Deteksi Otomatis oleh Engine KARSA

Ketika halaman HTML dimuat (DOM Content Loaded), pustaka `karsa.js` secara cerdas akan memindai seluruh halaman untuk mencari tag `<script>` yang memiliki atribut `type="text/karsa"`. Engine kemudian akan membaca seluruh kode (baik inline maupun dari file `.ks`), melakukan kompilasi on-the-fly lewat 5 tahap pipeline, dan mengeksekusinya langsung di browser.

BAB 3

Variabel & Tipe Data Dasar

3.1 Wadah Penyimpanan Data

Variabel adalah wadah untuk menyimpan data di dalam memori komputer yang dapat Anda gunakan dan modifikasi dalam program. Di dalam KARSA, deklarasi variabel dirancang dengan filosofi yang sangat mendalam karena pilihan jenis variabel akan langsung memengaruhi bagaimana data Anda berinteraksi dengan antarmuka web.

Dalam KARSA, terdapat **tiga kata kunci khusus** untuk mendeklarasikan variabel. Setiap kata kunci memiliki sifat reaktivitas (reactivity) dan mutabilitas (mutability) yang berbeda.

3.2 Perintah data — Variabel Reaktif

Kata kunci data digunakan untuk mendeklarasikan variabel reaktif. Perubahan nilai pada variabel jenis ini akan dipantau oleh program secara otomatis.

Jika variabel reaktif ini Anda hubungkan (binding) ke dalam elemen DOM di layar, maka begitu nilainya diubah, **tampilan di layar akan otomatis diperbarui** tanpa perlu Anda perbarui manual!

Deklarasi dan Modifikasi Variabel Reaktif

```
--! Mendeklarasikan variabel reaktif
data penghitung = 0
data namaPengguna = "Budi Santoso"

--! Mengubah nilai variabel reaktif menggunakan perintah 'simpan' atau 'tambahkan'
tambahkan 1 ke penghitung
simpan "Andi Wijaya" ke namaPengguna
```

Di Balik Layar Kompilasi Variabel Reaktif

Ketika Anda menulis `data penghitung = 0`, **Compiler** KARSA mengubahnya menjadi kode JavaScript: `const penghitung = __createReactive(0);`. Sistem membungkus nilai 0 dalam objek Proxy JavaScript yang memiliki setter dan getter. Begitu Anda mengeksekusi

tambahkan 1 ke penghitung (`__setState(penghitung, penghitung.value + 1)`), objek Proxy akan mengirimkan notifikasi kepada seluruh elemen DOM yang memakai variabel tersebut untuk render ulang segera.

3.3 Perintah tetap — Konstanta Immutable

Kata kunci `tetap` mendeklarasikan sebuah konstanta yang nilainya tidak dapat diubah (`immutable` / `read-only`) setelah diinisialisasi pertama kali. Gunakan kata kunci ini untuk data tetap seperti konfigurasi, nama aplikasi, atau rumus tetap.

Deklarasi Konstanta Immutable

```
tetap FAKTOR_PAJAK = 0.11
tetap NAMA_APLIKASI = "Sistem Manajemen Keuangan KARSA"

--! ERROR! Perintah di bawah ini akan memicu Error E3003 (Mencoba merotasi konstanta)
simpan 0.15 ke FAKTOR_PAJAK
```

3.4 Perintah ubah — Variabel Biasa (Non-Reaktif)

Kata kunci `ubah` digunakan untuk mendeklarasikan variabel biasa (`mutable`) yang **bersifat non-reaktif**. Perubahan nilainya tidak akan dipantau oleh sistem reaktivitas UI.

Gunakan variabel ini untuk keperluan internal proses logika (`internal state`), seperti variabel dalam penghitungan loop (`loop counter`), hasil sementara, atau pengolahan data murni yang tidak perlu memicu rendering ulang di layar.

Deklarasi Variabel Mutable Biasa

```
ubah totalSementara = 0
ubah indeksLoop = 1

simpan 500 ke totalSementara
```

Hasil Kompilasi Variabel Biasa

Perintah `ubah totalSementara = 0` akan dikompilasi secara sederhana dan ringan menjadi JavaScript vanilla murni: `let totalSementara = 0;`. Tidak ada pembungkusan Proxy sehingga eksekusinya berjalan secepat kilat.

3.5 Ringkasan Perbandingan Tiga Jenis Variabel

Kata Kunci	Reaktivitas	Mutabilitas	Padanan JS	Skenario Penggunaan Utama
data	☐ Reaktif	☐ Mutable	<code>const x = Proxy</code>	Penyimpanan state utama aplikasi yang langsung terhubung ke elemen antarmuka DOM.
tetap	☐ Non-Reaktif	☐ Immutable	<code>const x = nilai</code>	Penyimpanan konstanta, URL API, kunci rahasia, atau objek referensi read-only.
ubah	☐ Non-Reaktif	☐ Mutable	<code>let x = nilai</code>	Penyimpanan state internal, variabel iterasi loop, penghitungan latar belakang yang cepat.

3.6 Pengecekan Tipe Opsional (Type Hinting)

KARSA memiliki fitur Type Hinting yang memungkinkan Anda mendefinisikan tipe data variabel secara eksplisit. Fitur ini sangat membantumu mencegah bugs lewat proses pemeriksaan oleh Analyzer di tahap 4 pipeline. Tuliskan tanda titik dua : setelah nama variabel.

Type Hinting di KARSA

```
data hargaAwal: angka = 150000
tetap STATUS_AKTIF: benar-salah = benar
ubah pesanSapaan: teks = "Halo Programmer!"

--! WARNING! Perintah di bawah ini akan memunculkan Peringatan W4001 (Type mismatch)
data umurBudi: angka = "Dua Puluh Lima"
```

3.7 Membedah Tipe Data Dasar

Dalam KARSA, seluruh tipe data dasar diadaptasi ke penamaan berbahasa Indonesia:

1. **Angka (angka)**: Menyimpan seluruh jenis bilangan, baik bulat (integer) maupun desimal (float). Contoh: 10, -5, 3.14, 0.75.
2. **Teks (teks)**: Menyimpan deretan karakter teks (string). Ditulis dengan diapit tanda kutip ganda atau tunggal. Contoh: "Dunia KARSA", 'Indonesia Coding'.
3. **Boolean (benar-salah)**: Hanya dapat diisi oleh dua kata kunci absolut: benar (true) atau salah (false).
4. **Array (array)**: Menyimpan kumpulan/koleksi data yang terurut berdasarkan indeks. Menggunakan tanda kurung siku []. Contoh: data daftarKota: array = ["Jakarta", "Surabaya", "Bandung", "Medan"].
5. **Objek (objek)**: Menyimpan kumpulan pasangan kunci dan nilai (key-value pairs). Menggunakan tanda kurung kurawal {}. Contoh: data profilSiswa: objek = { nama: "Siti", jurusan: "Informatika", ipk: 3.88 }.

BAB 4

Operator & Struktur Percabangan

4.1 Operator Matematika (Aritmatika)

KARSA menyediakan kata kunci berbahasa Indonesia yang sangat intuitif untuk seluruh operasi matematika dasar. Anda dapat menggunakan alias kata bahasa Indonesia atau simbol matematika aslinya.

Kata Kunci	Simbol	Contoh Penggunaan	Hasil Output	Deskripsi Operasi
<code>tambah</code>	+	15 tambah 5	20	Penjumlahan dua angka atau penyambungan teks (string concatenation).
<code>kurang</code>	-	20 kurang 8	12	Pengurangan angka pertama oleh angka kedua.
<code>kali</code>	*	6 kali 4	24	Perkalian dua angka.
<code>bagi</code>	/	30 bagi 5	6	Pembagian angka pertama oleh angka kedua.
<code>mod</code>	%	10 mod 3	1	Menghitung sisa hasil pembagian (modulo).
<code>pangkat</code>	**	3 pangkat 3	27	Memangkatkan angka pertama dengan angka kedua.

Contoh Operasi Aritmatika

```
data hargaSatuan = 25000
data jumlahBeli = 3
```

```
data diskon = 10000
```

```
data totalBayar = (hargaSatuan kali jumlahBeli) kurang diskon
tampilkan pesan "Total yang harus dibayar: Rp" + totalBayar
```

4.2 Operator Perbandingan

Operator perbandingan membandingkan dua nilai dan mengembalikan tipe data benar-salah (benar atau salah). KARSA secara ketat (strict) menggunakan perbandingan nilai dan tipe data sekaligus.

Kata Kunci Bahasa Indonesia	Simbol JS	Contoh Evaluasi	Hasil	Keterangan
sama dengan	===	10 sama dengan 10	benar	Identik secara nilai dan identik secara tipe data.
tidak sama dengan	!==	"A" tidak sama dengan "B"	benar	Nilai atau tipe data berbeda.
lebih dari	>	15 lebih dari 20	salah	Angka pertama lebih besar ketat dari angka kedua.
kurang dari	<	8 kurang dari 12	benar	Angka pertama lebih kecil ketat dari angka kedua.
paling sedikit	>=	10 paling sedikit 10	benar	Lebih besar atau sama dengan (greater than or equal).
paling banyak	<=	5 paling banyak 4	salah	Lebih kecil atau sama dengan (less than or equal).

4.3 Operator Logika

Operator logika menggabungkan dua atau lebih evaluasi kondisi:

1. **dan (&&):** Menghasilkan benar jika **seluruh kondisi** bernilai benar. Jika ada satu saja yang salah, hasilnya salah.
2. **atau (||):** Menghasilkan benar jika **minimal ada satu kondisi** yang bernilai benar.
3. **bukan (!):** Membalikkan logika. Jika kondisi benar, menjadi salah. Jika salah, menjadi benar.

Contoh Operasi Logika

```
data nilaiUjian = 85
data nilaiSikap = "A"

--! Evaluasi logika gabungan
data lulusKriteria = (nilaiUjian paling sedikit 75) dan (nilaiSikap sama dengan "A")
data dapatBeasiswa = (nilaiUjian lebih dari 90) atau (lulusKriteria sama dengan benar)
```

4.4 Struktur Alur Percabangan (Control Flow)

Untuk membuat aplikasi web Anda mampu membuat keputusan otomatis berdasarkan kondisi tertentu, KARSA menyediakan tiga kata kunci percabangan yang sangat elegan dan terstruktur:

1. **jika <kondisi>::** Perintah pembuka. Menjalankan blok kode di bawahnya jika kondisi bernilai benar.
2. **kalau <kondisi>::** Alternatif kondisi lanjutan (else if). Dievaluasi jika kondisi jika di atasnya salah.
3. **jika tidak::** Jalur terakhir (else). Dieksekusi jika seluruh kondisi jika dan kalau di atasnya bernilai salah.

Di dalam KARSA, blok percabangan menggunakan tanda titik dua : di akhir baris dan penulisan menjorok (indentation) yang rapi di bawahnya.

Sintaks Percabangan jika / kalau / jika tidak

```
data totalBelanja = 650000

jika totalBelanja paling sedikit 1000000:
  tampilkan pesan "Selamat! Anda mendapatkan Diskon Eksklusif 25%"
  simpan 0.25 ke faktorDiskon

kalau totalBelanja paling sedikit 500000:
  tampilkan pesan "Anda mendapatkan Diskon Spesial 10%"
  simpan 0.10 ke faktorDiskon

jika tidak:
  tampilkan pesan "Belanja sedikit lagi untuk mendapatkan diskon."
  simpan 0.0 ke faktorDiskon
```


BAB 5

Struktur Perulangan & Kontrol Loop

5.1 Mengotomatisasi Pekerjaan Berulang

Perulangan (looping) adalah mekanisme dasar pemrogram untuk menjalankan suatu blok kode secara berulang kali tanpa perlu menuliskannya secara manual baris demi baris.

KARSA menyediakan dua struktur utama untuk perulangan, yaitu `ulangi` dan `selama`, yang dirancang untuk menangani berbagai skenario logika program.

5.2 Struktur Perulangan `ulangi` (3 Variasi Elegan)

Kata kunci `ulangi` dirancang oleh KARSA agar memiliki fleksibilitas luar biasa dengan menghadirkan tiga variasi penulisan yang menyerupai bahasa natural:

a) Perulangan Sederhana N Kali

Variasi ini digunakan ketika Anda sudah tahu secara pasti berapa kali Anda ingin mengulang suatu blok eksekusi. Gunakan sintaks `ulangi <jumlah> kali:`.

Perulangan N Kali

```
ulangi 5 kali:
  tampilkan pesan "Perintah ini akan dikompilasi dan dicetak sebanyak lima kali."
  buat paragraf -> teks: "Hasilkan paragraf baru otomatis di layar."
```

b) Perulangan Koleksi / Array (Setara `forEach`)

Variasi ini digunakan untuk menjelajahi isi dari sebuah Array atau isi koleksi elemen secara berurutan (iteration). Gunakan sintaks `ulangi <item> dari <array>:`.

Perulangan Koleksi Array

```
data daftarBuah = ["Apel Malang", "Jeruk Pontianak", "Mangga Indramayu", "Pisang Ambon"]
```

```
ulangi buah dari daftarBuah:  
  tampilkan pesan "Mencetak buah Nusantara: " + buah  
  buat li -> teks: buah
```

c) Perulangan Rentang Angka (Setara For Loop JS)

Variasi ini digunakan saat Anda membutuhkan nomor pengindeks (loop index) yang bertambah secara bertahap dari satu titik ke titik lainnya. Gunakan sintaks `ulangi <indeks> dari <awal> sampai <akhir>:`

Perulangan Rentang Angka

```
ulangi nomor dari 1 sampai 10:  
  data hasilPerkalian = nomor kali 7  
  tampilkan pesan "7 x " + nomor + " = " + hasilPerkalian
```

Padanan JavaScript Kompilasi Ulangi Rentang

Kode `ulangi nomor dari 1 sampai 10:` akan diterjemahkan oleh **Compiler** KARSA secara presisi menjadi loop standar JS: `for (let nomor = 1; nomor <= 10; nomor++) { ... }.`

5.3 Struktur Perulangan selama (While Loop)

Kata kunci `selama` digunakan ketika Anda tidak tahu pasti berapa jumlah perulangan yang dibutuhkan, namun Anda ingin loop terus berjalan **selama suatu kondisi bernilai benar**. Gunakan sintaks `selama <kondisi>:`

Hati-hati dengan Infinite Loop

Saat menggunakan perulangan `selama`, pastikan Anda selalu memiliki mekanisme mutasi (state update) di dalam blok perulangan tersebut (seperti mengubah nilai variabel `ubah`). Jika kondisi selalu bernilai benar dan tidak pernah menjadi salah, browser Anda akan membeku karena mengalami perulangan tak terbatas (infinite loop / Error E6002).

Perulangan selama (While Loop)

```
ubah bensinTersedia = 5
```

```
selama bensinTersedia lebih dari 0:  
    tampilkan pesan "Mobil berjalan... Bensin tersisa: " + bensinTersedia  
    kurangi bensinTersedia dengan 1  
  
tampilkan pesan "Bensin habis! Mobil berhenti berjalan."
```

5.4 Kata Kunci Kontrol Perulangan (berhenti & lewati)

Di dalam sebuah blok perulangan ulangi maupun selama, terkadang Anda ingin mengambil tindakan khusus di tengah jalan. KARSA menghadirkan dua kata kunci kontrol:

1. **berhenti (Setara break)**: Digunakan untuk **menghentikan seluruh proses perulangan** saat itu juga dan keluar melompat dari blok loop secara paksa.
2. **lewati (Setara continue)**: Digunakan untuk **melewati sisa eksekusi pada putaran saat ini** saja, dan langsung melompat melanjutkan ke putaran perulangan berikutnya.

Penggunaan berhenti dan lewati

```
data nilaiSiswa = [60, 75, 40, 90, 88, 30, 95]  
  
ulangi nilai dari nilaiSiswa:  
    --! Jika nilai di bawah 50, abaikan dan lewati ke siswa berikutnya  
    jika nilai kurang dari 50:  
        lewati  
  
    --! Jika menemukan nilai sempurna, hentikan pencarian segera  
    kalau nilai sama dengan 95:  
        tampilkan pesan "Menemukan siswa berprestasi tinggi! Pencarian dihentikan."  
        berhenti  
  
jika tidak:  
    tampilkan pesan "Siswa lulus dengan nilai: " + nilai
```

BAB 6

Menulis Fungsi & Interoperabilitas JavaScript

6.1 Konsep Sub-Program dan Modularitas

Fungsi (function) adalah sekumpulan blok kode yang dikelompokkan dan diberi nama, sehingga dapat dipanggil secara berulang kali dari bagian program mana pun. Fungsi merupakan kunci utama untuk membangun aplikasi web yang modular, bersih, dan mudah dirawat.

Dalam KARSA, mendeklarasikan fungsi menggunakan kata kunci `fungsi`, diikuti dengan nama fungsi, tanda kurung yang berisi daftar parameter opsional, dan diakhiri tanda titik dua `:`.

Deklarasi dan Pemanggilan Fungsi

```
--! Mendeklarasikan fungsi sederhana dengan dua parameter
fungsi hitungLuasPersegiPanjang(panjang, lebar):
  data luas = panjang kali lebar
  tampilkan pesan "Luas area adalah: " + luas
  kembalikan luas

--! Memanggil fungsi yang telah dibuat
data area1 = hitungLuasPersegiPanjang(15, 8)
data area2 = hitungLuasPersegiPanjang(20, 10)
```

6.2 Mengembalikan Nilai (Keyword kembalikan)

Fungsi tidak hanya dapat mengeksekusi proses, tetapi juga dapat mengirimkan hasil pemrosesan tersebut kembali kepada pemanggilnya. Gunakan kata kunci `kembalikan` (Setara `return` dalam JS).

Begitu perintah `kembalikan` dieksekusi, sistem akan langsung keluar dari fungsi tersebut. Jika ada baris kode di bawah perintah `kembalikan` dalam satu fungsi, baris tersebut tidak akan pernah dieksekusi.

Mengembalikan Nilai dengan keyword kembalikan

```
fungsi verifikasiKelayakanDiskon(totalBeli, pelangganVIP):  
  jika pelangganVIP sama dengan benar:  
    --! Langsung kembalikan benar karena pelanggan VIP selalu dapat diskon  
    kembalikan benar  
  
  kalau totalBeli paling sedikit 500000:  
    kembalikan benar  
  
  jika tidak:  
    kembalikan salah
```

6.3 Fungsi dengan Nilai Bawaan (Default Parameter)

KARSA mendukung pengisian nilai bawaan pada parameter fungsi. Jika Anda memanggil fungsi tersebut tanpa mengisi argumen, maka sistem secara otomatis akan menggunakan nilai bawaan yang telah ditentukan.

Default Parameters di KARSA

```
--! Menentukan nilai bawaan langsung pada daftar parameter  
fungsi sapaPelanggan(nama = "Pengguna", levelMembership = "Reguler"):  
  buat judul -> teks: "Selamat Datang, " + nama  
  buat paragraf -> teks: "Tipe Akun: " + levelMembership  
  
--! Pemanggilan tanpa argumen akan memakai ("Pengguna", "Reguler")  
sapaPelanggan()  
  
--! Pemanggilan dengan argumen akan menimpa nilai bawaan  
sapaPelanggan("Rina Melati", "Platinum Premium")
```

6.4 Interoperabilitas Mulus dengan JavaScript Vanilla

Salah satu keunggulan terbesar dan terhebat KARSA adalah **tidak terisolasi dari dunia luar**. KARSA dibangun agar memiliki interoperabilitas langsung yang mulus dengan seluruh objek, API, dan fungsi bawaan JavaScript DOM browser maupun libraries eksternal.

Anda dapat memanggil API standar browser secara natural langsung dari dalam logika kode .ks Anda!

JS Interop Langsung di KARSA

```
--! Menggunakan objek Math bawaan JavaScript  
data angkaAcak = Math.random()  
data hasilBulat = Math.floor(4.85)
```

```
--! Memanggil API Browser standar seperti alert, confirm, dan console.log
console.log("Pesan debugging ini dikirim langsung dari dalam skrip KARSA.")
alert("Halo! Ini memanggil dialog alert JS murni.")
```

```
--! Menjalankan Web Storage API (localStorage)
localStorage.setItem("karsa_sesi_pengguna", "Aktif_v031")
data sesiLama = localStorage.getItem("karsa_sesi_pengguna")
```

Memanggil Pustaka Pihak Ketiga (Misal SweetAlert / Axios / Chart.js)

Jika di dalam HTML Anda telah memuat pustaka pihak ketiga, misalnya `<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>`, maka di dalam file `app.ks`, Anda dapat langsung menulis: `Swal.fire({ title: "Hebat!", text: "KARSA berhasil memanggil SweetAlert!", icon: "success" })`

BAB 7

Output DOM & Penciptaan Elemen dengan buat

7.1 Menguasai Mekanisme Output Teks via tampilkan

Pernyataan `tampilkan` adalah perintah mendasar di dalam KARSA untuk mencetak teks atau hasil kalkulasi data secara visual ke layar halaman web.

Dalam arsitektur KARSA v0.3.1, perintah `tampilkan` dirancang memiliki **5 Mode Output** yang sangat canggih untuk memberikan kontrol penuh tentang bagaimana teks disisipkan ke dalam elemen DOM.

Kata Kunci Mode	Sintaksis Lengkap	Padanan Target JS	Mekanisme & Keterangan Internal
<code>ganti</code>	<code>tampilkan ganti #pesan "Halo"</code>	<code>target.innerText = "Halo"</code>	Menghapus seluruh teks asli yang ada pada elemen target, lalu menggantinya dengan teks baru (Mode Bawaan / Overwrite).
<code>tambahkan</code>	<code>tampilkan tambahkan #p "A"</code>	<code>target.innerText += "A"</code>	Menambahkan teks baru tepat di belakang akhir teks yang sudah ada di elemen target (Append).
<code>awalan</code>	<code>tampilkan awalan #p "A"</code>	<code>target.innerText = "A" + ...</code>	Menyisipkan teks baru tepat di posisi paling depan / sebelum teks asli elemen target (Prepend).

sebelum

tampilkan sebelum #p
"A"

target.insertAdjacentHTML

Menyisipkan teks baru sebagai elemen independen baru yang posisinya berada tepat di luar sebelum elemen target (Before).

sesudah

tampilkan sesudah #p
"A"

target.insertAdjacentHTML

Menyisipkan teks baru sebagai elemen independen baru yang posisinya berada tepat di luar sesudah elemen target (After).

7.2 Membangun Antarmuka Penuh dengan Perintah buat

Selain sekadar menyisipkan teks, KARSA memungkinkan Anda membangun seluruh struktur hierarki elemen HTML murni (seperti form, tabel, input, kartu) secara dinamis menggunakan pernyataan **buat**.

Sintaks dasar: `buat <tag_alias>#<id>.<kelas> -> <kunci_properti>: <nilai>, <kunci2>: <nilai>`

Membangun Elemen Web dengan Pernyataan buat

```
--! Membangun sebuah Kartu Profil lengkap dengan elemen anaknya
buat ruang#kartuProfil.kartu-elegan -> gaya: "padding: 20px; border: 1px solid #ccc;":
  buat judul -> teks: "Profil Ahli Web Indonesia"
  buat paragraf.biodata -> teks: "Spesialis Pengembangan Antarmuka dengan KARSA."
  buat gambar#avatar -> sumber: "foto-ahli.jpg", alt: "Foto Profil Ahli", lebar: "120"
  buat tombol#tombolKontak.btn-utama -> teks: "Hubungi Sekarang"
```

7.3 Referensi Master 20+ Tag Alias Bahasa Indonesia

KARSA menstandarisasi penamaan elemen HTML ke dalam kosa kata Indonesia baku yang sangat indah:

Alias Tag	HTML	Keterangan Fungsi	Alias Tag	HTML	Keterangan Fungsi
-----------	------	-------------------	-----------	------	-------------------

ruang	<code><div></code>	Wadah pembungkus blok (Container)	masukan	<code><input></code>	Kolom isian interaktif / input form
paragraf	<code><p></code>	Blok paragraf teks biasa	area-teks	<code><textarea></code>	Kolom input teks multi-baris
judul	<code><h1></code>	Judul halaman utama (Level 1)	formulir	<code><form></code>	Wadah pengiriman data form
judul2	<code><h2></code>	Sub-judul bab (Level 2)	tabel	<code><table></code>	Wadah utama data tabel
judul3	<code><h3></code>	Sub-judul sub-bab (Level 3)	baris-tabel	<code><tr></code>	Baris horizontal dalam tabel
tombol	<code><button></code>	Tombol interaktif yang bisa diklik	sel-tabel	<code><td></code>	Sel data spesifik dalam baris tabel
gambar	<code></code>	Penampil file gambar visual	daftar	<code></code>	Kumpulan daftar dengan poin bulatan
tautan	<code><a></code>	Tautan / hyperlink ke URL lain	item	<code></code>	Item spesifik di dalam daftar

7.4 Padanan Kunci Properti Elemen

Ketika Anda mengisi properti setelah tanda panah `->`, KARSA memetakannya secara mulus ke JS:

- teks: `"..."` dipetakan ke `element.innerText` (Teks aman, mengabaikan tag HTML).
- html: `"<b>...</b>"` dipetakan ke `element.innerHTML` (Mampu me-render tag HTML tebal/miring).

- nilai: ... dipetakan ke `element.value` (Untuk membaca atau mengisi isi dari elemen masukan).
- sumber: "..." dipetakan ke `element.src` (Alamat file pada tag gambar).
- tautan: "..." dipetakan ke `element.href` (Alamat URL tujuan pada tag tautan).
- gaya: "..." dipetakan ke `element.style.cssText` (Untuk styling elemen secara spesifik).

BAB 8

Interaktivitas Event Handling dengan ketika

8.1 Memberikan Nyawa pada Antarmuka (Event Driven)

Aplikasi web yang luar biasa bukanlah sekadar halaman dokumen statis yang diam, melainkan antarmuka hidup yang secara aktif merespons setiap sentuhan, ketikan, dan interaksi penggunaanya.

Dalam KARSA, mendengarkan dan merespons peristiwa (event handling) dilakukan secara elegan menggunakan pernyataan **ketika**.

8.2 Tiga Sintaksis Pemasangan Event

Terdapat tiga pola penulisan event handler yang didukung penuh oleh Compiler KARSA:

Pola 1: Pemasangan pada Elemen Spesifik (Target Selector)

Ditulis dengan menentukan selector elemen tujuan, diikuti kata kunci event.

Pemasangan Event Target Selector

```
buat tombol#tombolDaftar.btn-premium -> teks: "Daftar Sekarang"

--! Memasang event listener ke elemen ber-ID #tombolDaftar
ketika #tombolDaftar diklik:
  tampilkan ganti #pesanStatus "Proses Pendaftaran Sedang Berjalan..."
  simpan benar ke statusAktif
```

Pola 2: Self-Reference di Dalam Pernyataan buat

Ini adalah pola yang sangat disukai pengembang karena membuat struktur kode UI sangat padat dan hierarkis. Perintah **ketika** ditulis langsung sebagai anak di dalam blok **buat**. Tidak perlu repot menulis ID!

Pemasangan Event Self-Reference

```

buat ruang#kartuInteraktif.kartu:
  buat judul2 -> teks: "Arahkan Kursor Anda ke Sini"

  --! Pemasangan event mandiri yang melekat langsung ke #kartuInteraktif
  ketika diarahkan:
    perbarui properti -> gaya: "background-color: #f1f5f9; transform: scale(1.05);"
    buat p -> teks: "Kursor berhasil masuk ke area kartu!"

  ketika ditinggal-kursor:
    perbarui properti -> gaya: "background-color: #ffffff; transform: scale(1.0);"
    buat p -> teks: "Kursor telah pergi meninggalkan kartu."

```

Pola 3: Pemasangan Event Global pada Dokumen HTML

Digunakan untuk menangani peristiwa yang terjadi di tingkat halaman dokumen secara keseluruhan.

Pemasangan Event Dokumen Dimuat

```

ketika dimuat:
  console.log("Halaman HTML dan seluruh skrip KARSA telah selesai di-render dan siap dipakai.")
  perbaruiLayarAwal()

```

8.3 Tabel Master Referensi 13 Keyword Event KARSA

KARSA menghadirkan 13 kata kunci penanganan peristiwa berbahasa Indonesia yang mencakup seluruh skenario interaksi modern:

Keyword KARSA	Event Standar JS	Elemen Utama	Pemicu Peristiwa (Event Trigger) & Penjelasan
diklik	click	tombol, tautan, ruang	Pengguna melakukan klik kiri atau tap pada elemen tujuan.
diketik	input	masukan, area-teks	Terjadi setiap kali ada penambahan atau penghapusan karakter di kolom input.

disubmit	submit	formulir	Pengguna mengirimkan data form (menekan tombol submit atau enter).
diubah	change	masukan (tipe opsi)	Nilai pada elemen pilihan (select / checkbox / radio) berubah tetap.
ditekan	keydown	masukan, Global	Pengguna menekan sebuah tombol di keyboard komputer.
dilepas	keyup	masukan, Global	Pengguna melepaskan tekanan dari sebuah tombol keyboard.
difokus	focus	masukan, elemen aktif	Elemen antarmuka mendapatkan fokus aktif (diklik atau via tombol Tab).
ditinggal	blur	masukan, elemen aktif	Elemen antarmuka kehilangan fokus aktifnya.
diarahkan	mouseover	ruang, gambar, antarmuka	Kursor mouse pengguna melintasi masuk ke dalam batas area elemen.
ditinggal-kursor	mouseout	ruang, antarmuka	Kursor mouse pengguna keluar meninggalkan batas area elemen.
digulir	scroll	Halaman / ruang	Pengguna menggulir (scrolling) area halaman atau wadah dengan scrollbar.
dipasang	DOMInserted	Semua Komponen	Elemen baru saja berhasil dipasang (mounted) ke dalam pohon DOM antarmuka.
dilepas-dari-dom	DOMRemoved	Semua Komponen	Elemen dicabut / dihapus secara paksa dari pohon DOM antarmuka.

BAB 9

Komentar, Dokumentasi & Membedah Pesan Error

9.1 Menjaga Keterbacaan dan Dokumentasi Kode

Menulis kode bukan sekadar membuat program yang bisa dimengerti dan dijalankan oleh komputer, tetapi juga membuat dokumentasi yang dapat dibaca dan dipahami oleh manusia (baik rekan satu tim maupun Anda sendiri di masa depan).

KARSA menyediakan dua pola dokumentasi kode yang sangat rapi:

1. Komentar Sebaris (Single-line Comment)

Menggunakan tanda dua strip dan tanda seru `--!`. Seluruh teks yang berada di belakang tanda ini pada baris yang sama akan sepenuhnya diabaikan oleh Kompilator KARSA.

Komentar Sebaris

```
--! Ini adalah komentar sebaris untuk memberi penjelasan singkat  
data angkaKas = 5000000 --! Menyimpan saldo awal kas perusahaan
```

2. Komentar DocString Premium (Multi-line Documentation)

Fitur unik di KARSA v0.3.1. Diawali dengan tanda `--? [[` dan ditutup dengan `]]`. Ini digunakan untuk menulis spesifikasi teknis mendalam, penjelasan fungsi, atau keterangan modul.

Komentar DocString Premium

```
--? [[  
Modul Pemroses Transaksi Finansial v1.0  
Dibuat oleh: Tim Arsitek KARSA  
Tanggung Jawab: Melakukan validasi limit transaksi, menghitung potongan biaya admin,  
serta mencatat log transaksi ke dalam sistem pemantauan audit.  
]]  
fungsi verifikasiTransaksi(jumlah): ...
```

9.2 Membedah Sistem Error KARSA (E1xxx - E6xxx)

KARSA memiliki sistem penanganan error terlengkap yang langsung memetakan kendala ke tahap pipeline yang bersangkutan. Berikut adalah rincian Master Error Codes di dalam KARSA:

Kode Error	Kategori / Tahap	Contoh Pemicu Fatal	Saran & Solusi Perbaikan Mandiri
E1001	Lexer (Tahap 1)	Karakter asing tak dikenal di luar kosa kata, misal menggunakan simbol @ atau \$.	Periksa kembali teks sumber. Pastikan Anda hanya menggunakan karakter, operator, dan kata kunci baku bahasa Indonesia KARSA.
E2001	Parser (Tahap 2)	Hilangnya tanda penutup sintaksis, misal lupa menaruh tanda kurung penutup atau blok tanpa titik dua :.	Periksa kelengkapan pasangan tanda kurung (), [], {} dan pastikan blok pembuka jika, ulangi diakhiri dengan tanda titik dua :.
E3001	Resolver (Tahap 3)	Mencoba memanggil variabel yang belum pernah dideklarasikan, misal langsung menulis <code>simpan 10 ke x</code> tanpa <code>data x</code> .	Lakukan deklarasi secara formal terlebih dahulu menggunakan kata kunci <code>data</code> , <code>tetap</code> , atau <code>ubah</code> sebelum melakukan modifikasi nilai.
E3003	Resolver (Tahap 3)	Rotasi paksa konstanta, yaitu mencoba merombak nilai pada variabel yang dideklarasikan dengan kata kunci <code>tetap</code> .	Konstanta bersifat <code>read-only</code> . Jika data tersebut memang dirancang untuk terus berubah seiring waktu, ganti deklarasi <code>tetap</code> menjadi <code>data</code> atau <code>ubah</code> .
E4001	Analyzer (Tahap 4)	Konflik tipe data (Type Mismatch), yaitu mengisi variabel angka dengan nilai teks: <code>data umur: angka = "Tua"</code> .	Patuhi aturan Type Hinting. Jika variabel didefinisikan sebagai <code>: angka</code> , isi dengan bilangan bulat atau desimal murni tanpa tanda kutip.

E5001	Compiler (Tahap 5)	Kegagalan internal pembentukan Abstract Syntax Tree saat terjemahan ke Vanilla JS.	Hal ini jarang terjadi. Coba sederhanakan struktur logika hierarki Anda atau pecah ekspresi yang terlalu rumit menjadi beberapa langkah variabel.
E6001	Runtime (Eksekusi)	Mencoba membagi sebuah angka dengan angka nol: <code>data x = 50</code> bagi <code>0</code> (Division by Zero).	Di dalam logika fungsi Anda, selalu pasang blok pemeriksaan sebelum melakukan pembagian: jika pembagi tak sama dengan <code>0</code> :

Strategi Cepat Menyelesaikan Error

1. **Tenang dan Baca Kode Error:** Setiap pesan error di console browser selalu menampilkan kode spesifik (misal Error E2002: AST Generation Failed at line 14).
2. **Lihat Nomor Baris:** Langsung buka file `.ks` Anda dan tuju nomor baris yang ditunjuk.
3. **Periksa Karakter Sekitar:** Seringkali error disebabkan oleh hal remeh seperti kurangnya titik desimal, kelupaan menaruh kutip string, atau kesalahan ketik nama variabel.

BAB 10

Mini-Proyek: Kalkulator Interaktif Lengkap

10.1 Merakit Karya Master Aplikasi Web Anda

Kini saatnya menyatukan 100% seluruh teori, fondasi, kata kunci, dan konsep yang telah Anda pelajari (variabel, reaktivitas, operator, alur kontrol, fungsi, penciptaan DOM buat, hingga event handler ketika) ke dalam sebuah **Proyek Nyata Premium**.

Kita akan membangun sebuah **Kalkulator Web Interaktif** bergaya modern dengan fitur lengkap: penjumlahan, pengurangan, perkalian, pembagian, perhitungan persentase mod, tombol pembersihan (Clear), dan tombol penghapusan karakter Mundur (Backspace), lengkap dengan perlindungan penanganan error pembagian dengan nol!

10.2 Entry Point — kalkulator.html

File HTML ini bertugas sebagai wadah penyedia kerangka tata letak visual (UI Layout) dengan gaya CSS yang sangat menawan bergaya glassmorphism dan gradien khas KARSA.

kalkulator.html (UI Kerangka visual)

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Kalkulator Interaktif Premium KARSA</title>
  <style>
    - { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      display: flex; justify-content: center; align-items: center;
      min-height: 100vh;
      background: linear-gradient(135deg, #0f172a 0%, #1e293b 50%, #0f766e 100%);
    }
    .kalkulator {
      background: rgba(255, 255, 255, 0.95);
      padding: 25px; border-radius: 20px;
      box-shadow: 0 20px 60px rgba(0, 0, 0, 0.4);
    }
  </style>

```

```

        width: 340px;
        backdrop-filter: blur(10px);
    }
    .kalkulator h2 {
        text-align: center; color: #0f172a; margin-bottom: 18px;
        font-size: 1.6rem; font-weight: 700;
    }
    #layar {
        width: 100%; padding: 18px; font-size: 2.2rem; text-align: right;
        border: 2px solid #cbd5e1; border-radius: 12px; margin-bottom: 20px;
        background: #f8fafc; color: #0f172a; font-family: 'DejaVu Sans Mono', monospace;
        font-weight: bold; letter-spacing: 2px;
    }
    .tombol-grid {
        display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px;
    }
    button {
        padding: 20px; font-size: 1.3rem; border: none; border-radius: 12px;
        cursor: pointer; background: #f1f5f9; transition: all 0.2s cubic-bezier(0.4,0,0.2,1);
        font-weight: 600; color: #334155;
    }
    button:hover {
        transform: translateY(-3px); box-shadow: 0 8px 20px rgba(0,0,0,0.15);
        background: #e2e8f0;
    }
    button:active { transform: translateY(0px); }
    button.operasi { background: #d97706; color: white; font-size: 1.5rem; }
    button.operasi:hover { background: #b45309; }
    button.sama-dengan { background: #0f766e; color: white; grid-column: span 2; font-size:
1.6rem; }
    button.sama-dengan:hover { background: #115e59; }
    button.danger { background: #ef4444; color: white; font-weight: bold; }
    button.danger:hover { background: #dc2626; }
    #pesan-error {
        text-align: center; margin-top: 18px; padding: 10px; font-size: 0.95rem;
        border-radius: 8px; font-weight: 600; min-height: 42px; display: flex;
        align-items: center; justify-content: center;
    }
    .error-aktif { background: #fef2f2; color: #b91c1c; border: 1px solid #f87171; }
    .sukses-aktif { background: #ecfdf5; color: #047857; border: 1px solid #6ee7b7; }
</style>
</head>
<body>

<!-- Wadah Utama Aplikasi -->
<div class="kalkulator">
    <h2>Kalkulator KARSA</h2>

    <!-- Layar Output Reaktif -->
    <input type="text" id="layar" value="0" readonly>

    <!-- Grid Tombol Operasional -->
    <div class="tombol-grid">
        <button id="clear" class="danger">C</button>
        <button id="hapus">⌫</button>
        <button id="persen" class="operasi">%</button>
        <button id="bagi" class="operasi">÷</button>

        <button id="tujuh">7</button>
        <button id="delapan">8</button>
        <button id="sembilan">9</button>
        <button id="kali" class="operasi">×</button>

        <button id="empat">4</button>

```

```
<button id="lima">5</button>
<button id="enam">6</button>
<button id="kurang" class="operasi">-</button>

<button id="satu">1</button>
<button id="dua">2</button>
<button id="tiga">3</button>
<button id="tambah" class="operasi">+</button>

<button id="nol">0</button>
<button id="titik">.</button>
<button id="hitung" class="sama-dengan">=</button>
</div>

<!-- Panel Output Status / Error -->
<p id="pesan-error"></p>
</div>

<!-- Pustaka Engine Kompilator KARSA -->
<script src="engine/karsa.js"></script>

<!-- Pemanggilan Logika Skrip Eksternal KARSA -->
<script type="text/karsa" src="kalkulator.ks"></script>
</body>
</html>
```

10.3 Master Logika Reaktif — kalkulator.ks

Berikut adalah keseluruhan logika program yang ditulis 100% menggunakan kata kunci bahasa Indonesia dalam file berekstensi .ks. File ini mengelola state reaktif, memproses input setiap digit, dan mengeksekusi kalkulasi.

kalkulator.ks (Master Logika Aplikasi)

```
--? [[
Kalkulator Web Interaktif KARSA v0.3.1 Premium
Fitur: Penjumlahan, Pengurangan, Perkalian, Pembagian, Persentase Mod, Reset, Backspace.
Proteksi: Perlindungan pembagian dengan nol, pencegahan desimal ganda, batas limit layar.
]]

--! =====
--! 1. DEKLARASI VARIABEL STATE UTAMA (REAKTIF & INTERNAL)
--! =====
data teksLayar = "0"           --! Hubungan langsung ke elemen antarmuka #layar
ubah angkaPertama = kosong    --! Menyimpan memori operand pertama
ubah operatorAktif = kosong    --! Menyimpan memori operator matematika yang ditekan
ubah modeTungguAngka2 = salah --! Bendera penentu apakah sistem sedang menunggu angka ke-2

--! =====
--! 2. FUNGSI SINKRONISASI & UTILITAS
--! =====
--? [[
Fungsi: perbaruiTampilanLayar
Tanggung Jawab: Menyinkronkan nilai variabel teksLayar ke dalam elemen HTML ber-ID #layar.
]]
fungsi perbaruiTampilanLayar():
  perbarui #layar -> nilai: teksLayar

--? [[
Fungsi: resetSeluruhSistem (Setara aksi tombol C)
Tanggung Jawab: Mereset seluruh variabel memori dan mengosongkan panel status error.
]]
fungsi resetSeluruhSistem():
  simpan "0" ke teksLayar
  simpan kosong ke angkaPertama
  simpan kosong ke operatorAktif
  simpan salah ke modeTungguAngka2
  perbarui #pesan-error -> teks: "Sistem berhasil di-reset.", kelas: "sukses-aktif"
  perbaruiTampilanLayar()

--? [[ Fungsi Pembantu Penampil Pesan Status ]]
fungsi cetakPesanError(keterangan):
  perbarui #pesan-error -> teks: keterangan, kelas: "error-aktif"

fungsi cetakPesanSukses(keterangan):
  perbarui #pesan-error -> teks: keterangan, kelas: "sukses-aktif"

--! =====
--! 3. FUNGSI PENGELOLAAN INPUT PENGGUNA
--! =====
--? [[
Fungsi: terimaInputDigit
Tanggung Jawab: Menerima tombol digit (0-9), menghapus angka 0 di depan,
serta mereset layar otomatis jika baru saja menekan tombol operator.
```

```

]]
fungsi terimaInputDigit(digitBaru):
    --! Bersihkan panel pesan error setiap kali pengguna mengetik input baru
    perbarui #pesan-error -> teks: "", kelas: ""

    jika modeTungguAngka2 sama dengan benar:
        simpan digitBaru ke teksLayar
        simpan salah ke modeTungguAngka2
        perbaruiTampilanLayar()
        kembalikan

    kalau teksLayar sama dengan "0":
        simpan digitBaru ke teksLayar
        perbaruiTampilanLayar()
        kembalikan

    jika tidak:
        --! Batasi agar pengguna tidak memasukkan angka melebihi kapasitas 12 karakter
        jika teksLayar.panjang paling banyak 11:
            simpan (teksLayar + digitBaru) ke teksLayar
            perbaruiTampilanLayar()
        jika tidak:
            cetakPesanError("Batas maksimum 12 digit tercapai.")

--? [[ Menangani penekanan tombol titik desimal (.) ]]
fungsi terimaInputTitikDesimal():
    perbarui #pesan-error -> teks: "", kelas: ""

    jika modeTungguAngka2 sama dengan benar:
        simpan "0." ke teksLayar
        simpan salah ke modeTungguAngka2
        perbaruiTampilanLayar()
        kembalikan

    --! Cek apakah di layar sudah terdapat titik. Jika belum ada, baru tambahkan
    jika bukan (teksLayar ada di "."):
        simpan (teksLayar + ".") ke teksLayar
        perbaruiTampilanLayar()

--? [[ Menangani penekanan tombol Mundur / Backspace (⌫) ]]
fungsi jalankanPenghapusanMundur():
    perbarui #pesan-error -> teks: "", kelas: ""

    jika teksLayar.panjang lebih dari 1:
        simpan teksLayar.potong(0, teksLayar.panjang - 1) ke teksLayar
    jika tidak:
        simpan "0" ke teksLayar

    perbaruiTampilanLayar()

--! =====
--! 4. FUNGSI PROSES KALKULASI MATEMATIKA
--! =====
--? [[
Fungsi: kalkulasiInti
Melakukan perhitungan dua angka berdasarkan simbol operator yang tersimpan.
Telah dilengkapi proteksi penolakan pembagian dengan nol (Error E6001).
]]
fungsi kalkulasiInti(num1, num2, op):
    jika op sama dengan "+": kembalikan (num1 tambah num2)
    kalau op sama dengan "-": kembalikan (num1 kurang num2)
    kalau op sama dengan "×": kembalikan (num1 kali num2)
    kalau op sama dengan "÷":
        jika num2 sama dengan 0:

```

```

        cetakPesanError("[FATAL] Kesalahan: Pembagian dengan nol ditolak!")
        kembalikan 0
    kembalikan (num1 bagi num2)
    kalau op sama dengan "%": kembalikan (num1 mod num2)
    kembalikan num2

--? [[ Menangani penekanan tombol operator (+, -, ×, ÷, %) ]]
fungsi pasangOperatorMatematika(simbolOperatorBaru):
    perbarui #pesan-error -> teks: "", kelas: ""
    data nilaiLayarSaatIni = angka(teksLayar)

    jika operatorAktif sama dengan kosong:
        simpan nilaiLayarSaatIni ke angkaPertama

    kalau modeTungguAngka2 sama dengan salah:
        data hasilKalkulasi = kalkulasiInti(angkaPertama, nilaiLayarSaatIni, operatorAktif)
        simpan teks(hasilKalkulasi) ke teksLayar
        simpan hasilKalkulasi ke angkaPertama
        perbaruiTampilanLayar()

    simpan benar ke modeTungguAngka2
    simpan simbolOperatorBaru ke operatorAktif
    cetakPesanSukses("Operator " + simbolOperatorBaru + " siap.")

--? [[ Menangani penekanan tombol Sama Dengan (=) ]]
fungsi eksekusiHasilAkhir():
    perbarui #pesan-error -> teks: "", kelas: ""

    jika angkaPertama bukan kosong dan operatorAktif bukan kosong:
        data hasilAkhir = kalkulasiInti(angkaPertama, angka(teksLayar), operatorAktif)
        simpan teks(hasilAkhir) ke teksLayar
        simpan kosong ke angkaPertama
        simpan kosong ke operatorAktif
        simpan benar ke modeTungguAngka2
        cetakPesanSukses("◆ Perhitungan sukses!")
        perbaruiTampilanLayar()
    jika tidak:
        cetakPesanError("[PERHATIAN] Masukkan angka dan operator terlebih dahulu.")

--! =====
--! 5. KETIKA EVENT LISTENERS – MENGHUBUNGKAN ANTARMUKA & LOGIKA
--! =====

ketika #clear diklik: resetSeluruhSistem()
ketika #hapus diklik: jalankanPenghapusanMundur()

ketika #nol diklik: terimaInputDigit("0")
ketika #satu diklik: terimaInputDigit("1")
ketika #dua diklik: terimaInputDigit("2")
ketika #tiga diklik: terimaInputDigit("3")
ketika #empat diklik: terimaInputDigit("4")
ketika #lima diklik: terimaInputDigit("5")
ketika #enam diklik: terimaInputDigit("6")
ketika #tujuh diklik: terimaInputDigit("7")
ketika #delapan diklik: terimaInputDigit("8")
ketika #sembilan diklik: terimaInputDigit("9")

ketika #titik diklik: terimaInputTitikDesimal()

ketika #tambah diklik: pasangOperatorMatematika("+")
ketika #kurang diklik: pasangOperatorMatematika("-")
ketika #kali diklik: pasangOperatorMatematika("×")
ketika #bagi diklik: pasangOperatorMatematika("÷")
ketika #persen diklik: pasangOperatorMatematika("%")

```

```
ketika #hitung diklik: eksekusiHasilAkhir()  
  
--! Jalankan sinkronisasi inisialisasi begitu skrip dibaca pertama kali  
resetSeluruhSistem()
```

Latihan Soal Uji Pemahaman

Kerjakan latihan berikut untuk menguji dan memantapkan penguasaan Anda terhadap seluruh konsep di Modul 1. Soal-soal ini disusun secara bertingkat dari tingkat kesulitan mudah hingga analisis mendalam.

Soal Mudah (Penguasaan Teori & Filosofi)

1. **Pilar Filosofi:** Jelaskan apa yang dimaksud dengan zero-configuration dalam ekosistem KARSA. Mengapa pilar ini sangat penting dalam memajukan pendidikan pemrograman di Indonesia?
2. **Perbandingan Variabel:** Sebutkan perbedaan paling kritis antara kata kunci `data` dengan `ubah`. Jika Anda ingin menyimpan skor pertandingan yang harus langsung ditampilkan dan update di layar UI, kata kunci mana yang wajib Anda pakai dan mengapa?
3. **Mekanisme AST:** Sebutkan lima tahapan dalam Pipeline KARSA secara berurutan. Apa yang dilakukan oleh internal sistem pada tahap ke-3 (Resolver)?
4. **Transformasi Operator:** Tuliskan operator KARSA yang menjadi padanan presisi dari operator JavaScript berikut: `===`, `!==`, `>=`, `&&`, dan `**`.

Soal Sedang (Keterampilan Menulis Logika & Praktik)

5. **Deklarasi & Type Hint:** Tuliskan kode KARSA yang benar untuk mendeklarasikan:
 - Sebuah konstanta dengan nama `BIAYA_ADMIN` yang bernilai 2500 dan bertipe angka.
 - Sebuah variabel reaktif bernama `statusKelulusan` yang bernilai benar dan bertipe benar-salah.
 - Sebuah variabel biasa bernama `namaModul` dengan nilai "Modul 1 Fondasi".
6. **Struktur Percabangan Lengkap:** Tuliskan sebuah program singkat dalam KARSA yang mengevaluasi variabel `nilaiUjian` (bertipe angka).
 - Jika nilai bernilai paling sedikit 85, tampilkan pesan "Predikat A - Sangat Memuaskan".
 - Kalau nilai bernilai paling sedikit 70, tampilkan pesan "Predikat B - Memuaskan".
 - Jika tidak, tampilkan pesan "Predikat C - Harap Mengulang".
7. **Variasi Perulangan:** Tuliskan dua versi kode perulangan yang mencetak deretan angka dari 1 sampai 5. Versi pertama menggunakan struktur `ulangi ... dari ... sampai ...`, dan versi kedua menggunakan struktur `selama`.

Soal Sulit (Analisis Debugging Error & Mini Kasus)

8. **Analisis Error Compiler:** Perhatikan cuplikan potongan kode di bawah ini. Temukan dan jelaskan minimal dua kesalahan fatal yang akan memicu terjadinya error saat kompilasi atau eksekusi, serta sebutkan kode error yang akan dimunculkan oleh Compiler:

Kasus Soal Nomor 8

```
tetap bahasaPilihan = "KARSA Script"
simpan "JavaScript" ke bahasaPilihan

ubah hitungPutaran: angka = 10
data dataMasukan: angka = "Satu Juta"
```

9. **Beda Mekanisme Internal:** Bandingkan dan jelaskan dengan sangat rinci perbedaan hasil eksekusi JS dan efek visual antarmuka dari dua blok kode berikut:

Kasus Soal Nomor 9

```
--! Kode A:
data penghitung = 10
simpan 20 ke penghitung

--! Kode B:
ubah penghitung = 10
simpan 20 ke penghitung
```

10. **Kasus Proyek Nyata — Konverter Suhu Terstruktur:** Tuliskan skrip lengkap KARSA berekstensi .ks yang mengimplementasikan aplikasi konverter suhu Celcius ke Fahrenheit dan Kelvin. Program harus mendeklarasikan variabel Celcius (menggunakan data), menerapkan rumus tetap ($F = C \times 1.8 + 32$ dan $K = C + 273.15$) menggunakan konstanta tetap, dan mencetak seluruh hasilnya ke dalam elemen antarmuka DOM menggunakan mode output sesudah!

Solusi Latihan & Pembahasan Lengkap

Pembahasan Soal Mudah

1. **Zero-config** berarti pengembang web dapat langsung menulis dan menjalankan kode tanpa perlu menginstal runtime (seperti Node.js) atau bundler rumit di komputernya. Hal ini menghapus penghalang teknis awal (technical barrier to entry) sehingga siapa pun di Indonesia yang memiliki web browser dapat langsung belajar pemrograman web dalam hitungan detik.
2. Variabel `data` dibungkus di dalam mekanisme `Proxy` JavaScript yang memantau setiap modifikasi nilainya agar langsung memperbarui elemen antarmuka UI secara otomatis (Reaktif). Sebaliknya, `ubah` adalah variabel `let` biasa yang bersifat Non-Reaktif. Untuk menyimpan skor pertandingan yang harus langsung update di layar UI, **wajib menggunakan `data`** agar render ulang terjadi seketika saat gol tercetak.
3. Lima tahapan pipeline KARSA adalah: **1. Lexer, 2. Parser, 3. Resolver, 4. Analyzer, dan 5. Compiler**. Pada tahap ke-3 (Resolver), sistem bertugas memetakan seluruh identifikasi (nama variabel, fungsi, dan label) ke dalam tabel cakupan (Scope Table), serta memastikan tidak ada duplikasi nama variabel atau pemanggilan referensi yang terputus di dalam cakupan memori.
4. Padanan operator aslinya adalah:
 - `===` -> sama dengan
 - `!==` -> tidak sama dengan
 - `>=` -> paling sedikit
 - `&&` -> dan
 - `**` -> pangkat

Pembahasan Soal Sedang

5. Pembahasan sintaks deklarasi `ber-Type Hint`:

Solusi Soal Nomor 5

```
tetap BIAYA_ADMIN: angka = 2500
data statusKelulusan: benar-salah = benar
ubah namaModul: teks = "Modul 1 Fondasi"
```

6. Pembahasan sintaks percabangan:

Solusi Soal Nomor 6

```
data nilaiUjian = 78

jika nilaiUjian paling sedikit 85:
  tampilkan ganti #pesan "Predikat A - Sangat Memuaskan"
kalau nilaiUjian paling sedikit 70:
  tampilkan ganti #pesan "Predikat B - Memuaskan"
jika tidak:
  tampilkan ganti #pesan "Predikat C - Harap Mengulang"
```

7. Solusi dua variasi perulangan:

Solusi Soal Nomor 7

```
--! Versi 1: Struktur ulangi rentang angka
ulangi i dari 1 sampai 5:
  tampilkan tambahkan #daftarAngka (i + " ")

--! Versi 2: Struktur selama
ubah nomor = 1
selama nomor paling banyak 5:
  tampilkan tambahkan #daftarAngka (nomor + " ")
  tambahkan 1 ke nomor
```

Pembahasan Soal Sulit (Analisis Mendalam)

8. Analisis kesalahan fatal pada skrip:

- **Error E3003 (Rotasi Konstanta Illegal):** Pada baris ke-2, eksekusi `simpan "JavaScript"` ke `bahasaPilihan` ditolak keras oleh Compiler karena `bahasaPilihan` dideklarasikan menggunakan kata kunci `tetap` (read-only).
- **Peringatan W4001 / Ketidakcocokan Tipe Data (Type Mismatch):** Pada baris ke-5, variabel `dataMasukan` diwajibkan bertipe angka lewat type hint : `angka`, namun diisi dengan nilai awalan bertipe teks berupa `"Satu Juta"`. Analyzer di tahap 4 pipeline akan memberikan peringatan tajam.

9. Perbedaan mendasar di balik layar Kompilasi & Reaktivitas:

- **Kode A (`data penghitung = 10; simpan 20 ke penghitung`):** Diterjemahkan menjadi objek Proxy reaktif: `const penghitung = __createReactive(10); __setState(penghitung, 20);`. Di sini, eksekusi `simpan` akan memicu Reactive Signal untuk memberitahukan seluruh elemen antarmuka web yang terhubung agar langsung melakukan re-render secara seketika.
- **Kode B (`ubah penghitung = 10; simpan 20 ke penghitung`):** Diterjemahkan secara sederhana dan polos menjadi variabel biasa: `let penghitung = 10; penghitung = 20;`. Tidak ada pembungkusan Proxy yang terjadi. Akibatnya, meskipun nilai di dalam memori internal RAM berubah menjadi 20, **tampilan di layar HTML akan tetap statis menampilkan angka 10 lama!**

10. Skrip Master Proyek Konverter Suhu Lengkap:

konverter-suhu.ks (Solusi Soal Nomor 10)

```
--? [[
Proyek Konverter Suhu Multi-Skala KARSA v0.3.1
Mengubah suhu input Celcius secara instan ke dalam skala Fahrenheit dan Kelvin.
]]

--! 1. Deklarasi State Reaktif & Konstanta Rumus
data suhuCelcius: angka = 30
tetap FAKTOR_KALI_F: angka = 1.8          --! 9 bagi 5
tetap KONSTANTA_TAMBAH_F: angka = 32
tetap KONSTANTA_KELVIN: angka = 273.15

--! 2. Kalkulasi Data Reaktif
data suhuFahrenheit = (suhuCelcius kali FAKTOR_KALI_F) tambah KONSTANTA_TAMBAH_F
data suhuKelvin = suhuCelcius tambah KONSTANTA_KELVIN

--! 3. Cetak Hasilnya ke dalam DOM Antarmuka menggunakan mode 'sesudah'
tampilkan ganti #judulAplikasi "Hasil Konversi Suhu untuk " + suhuCelcius + "°C"

tampilkan sesudah #judulAplikasi (
  "<div>" +
  "<p>Suhu dalam Fahrenheit: <b>" + suhuFahrenheit + "°F</b></p>" +
  "<p>Suhu dalam Kelvin: <b>" + suhuKelvin + " K</b></p>" +
  "</div>"
)
```

Lampiran — Referensi Cepat

Padanan Sintaks

Gunakan tabel Cheat Sheet Master ini sebagai panduan cepat saat Anda mengembangkan aplikasi web dengan KARSA v0.3.1 untuk melihat secara langsung padanan kodenya dalam JavaScript murni.

1. Padanan Kata Kunci Deklarasi & Mutasi Variabel

Sintaksis KARSA Script (.ks)	Target Kompilasi JavaScript Murni	Keterangan & Perilaku Internal
data x = 10	const x = __createReactive(10)	Penciptaan variabel reaktif berbasis Proxy DOM.
tetap PI = 3.14	const PI = 3.14	Penciptaan konstanta read-only absolut.
ubah y = 5	let y = 5	Penciptaan variabel biasa non-reaktif.
simpan 20 ke x	__setState(x, 20) / x = 20	Modifikasi / assignment nilai variabel.
tambahkan 5 ke x	__setState(x, x.value + 5)	Operasi increment otomatis pada memori.
kurangi x dengan 3	__setState(x, x.value - 3)	Operasi decrement otomatis pada memori.
sisipkan "A" ke daftar	daftar.value.push("A")	Operasi push elemen baru ke dalam Array reaktif.

2. Padanan Kata Kunci Alur Kontrol & Perulangan

Sintaksis KARSA Script (.ks)	Target Kompilasi JavaScript Murni	Keterangan & Perilaku Internal
jika x sama dengan 1:	<code>if (x === 1) { ... }</code>	Pernyataan evaluasi percabangan utama.
kalau x lebih dari 5:	<code>else if (x > 5) { ... }</code>	Pernyataan evaluasi percabangan lanjutan.
jika tidak:	<code>else { ... }</code>	Jalur penyelesaian akhir jika semua salah.
ulangi 5 kali:	<code>for (let __i = 0; __i < 5; __i++)</code>	Perulangan blok secara instan N putaran.
ulangi item dari daftar:	<code>for (let item of daftar)</code>	Penjelajahan isi elemen Array (for...of).
selama x kurang dari 10:	<code>while (x < 10) { ... }</code>	Perulangan bersyarat (while loop).
berhenti / lewati	<code>break; / continue;</code>	Pemutusan atau pelompatan putaran loop.

3. Padanan Manipulasi DOM & Event Interaktif

Sintaksis KARSA Script (.ks)	Target Kompilasi JavaScript Murni	Keterangan & Perilaku Internal
tampilkan ganti #teks "A"	<code>document.querySelector("#teks").innerHTML = "A";</code>	Menyisipkan teks secara aman ke dalam target.

buat tombol#klikAku:	<code>document.createElement("button"</code>	Menciptakan elemen antarmuka baru di memori.
ketika #klikAku diklik:	<code>element.addEventListener("click"</code>	Pemasangan pemicu respons interaktif pengguna.
-> teks: "Sapa"	<code>element.innerText = "Sapa"</code>	Pengisian properti teks dalam blok buat / perbarui.

4. Master Referensi Padanan Properti DOM (Indonesia vs JS Asli)

KARSA memungkinkan Anda mengakses seluruh properti atribut DOM dalam bahasa baku yang sangat natural:

Properti KARSA	JS DOM Asli	Properti KARSA	JS DOM Asli
<code>.panjang</code>	<code>.length</code>	<code>.nonaktif</code>	<code>.disabled</code>
<code>.nilai</code>	<code>.value</code>	<code>.anak</code>	<code>.children</code>
<code>.teks</code>	<code>.innerText</code>	<code>.induk</code>	<code>.parentElement</code>
<code>.html</code>	<code>.innerHTML</code>	<code>.fokus</code>	<code>.focus()</code>
<code>.tipe</code>	<code>.type</code>	<code>.atribut</code>	<code>.getAttribute()</code>
<code>.nama</code>	<code>.name</code>	<code>.gaya</code>	<code>.style.cssText</code>
<code>.ditandai</code>	<code>.checked</code>	<code>.sumber</code>	<code>.src</code>

Checklist Mandiri Pembelajaran Modul 1

Sebelum Anda melanjutkan perjalanan hebat menuju **Modul 2 (Reaktivitas & Komponen Lanjutan)**, gunakan daftar Checklist di bawah ini untuk memverifikasi secara jujur tingkat penguasaan dan kesiapan Anda. Beri tanda centang (✓) pada setiap poin yang sudah berhasil Anda pahami sepenuhnya.

Status	Kompetensi & Indikator Keterampilan Target	Bab Terkait
	Pemahaman Arsitektur & Pipeline: Mampu menyebutkan dan menjelaskan alur kerja 5 tahap internal (Lexer -> Parser -> Resolver -> Analyzer -> Compiler).	Bab 1
	Persiapan Zero-Config: Berhasil membuat file HTML murni yang menyertakan pustaka karsa.js dan memanggil file .ks eksternal secara tepat.	Bab 2
	Penguasaan Tiga Jenis Variabel: Tahu kapan saatnya wajib mendeklarasikan variabel dengan data (Reaktif), tetap (Konstanta), dan ubah (Biasa / Internal State).	Bab 3
	Penerapan Operator & Percabangan: Berhasil menggabungkan operator matematika dan perbandingan dalam struktur alur kontrol jika / kalau / jika tidak.	Bab 4
	Penggunaan Variasi Perulangan: Mampu menjelajahi koleksi Array dengan ulangi ... dari ... dan menjalankan loop bersyarat menggunakan struktur selama.	Bab 5
	Penciptaan Elemen Web & Interaktivitas: Telah lancar merakit elemen DOM dengan perintah buat serta merespons sentuhan klik pengguna via pernyataan ketika.	Bab 7 & 8
	Keberhasilan Perakitan Proyek Nyata Premium: Telah berhasil membuat, mengetik, mengompilasi, dan menguji coba aplikasi Kalkulator Interaktif Lengkap di browser tanpa error fatal.	Bab 10

PENUTUP MODUL 1

Selamat & Sukses!

Anda telah berhasil menuntaskan seluruh materi fundamental Modul 1 KARSA v0.3.1 Premium.

Kini Anda telah menguasai fondasi utama dalam merakit antarmuka web interaktif berbasis bahasa Indonesia. Pengetahuan ini adalah bekal terpenting untuk membangun aplikasi berskala korporat di masa depan.

◆ Lanjutkan Langkah ke Modul 2 — Logika & Kontrol Alur Karsa